

CAHIER D'IMPLÉMENTATION

Application Web DR-DOCSCOL

Justification des choix technologiques

Projet : Application de demande document académique
Contexte : Cameroun — OBC et DECC
Version : 1.0 — Juin 2026
Documents associés : Cahier d'analyse, Cahier de conception, Cahier des charges
Complément soutenance : docs/guide_technologies_modules_soutenance.md

TABLE DES MATIÈRES

- 1. Objet du cahier d'implémentation
- 2. Principes directeurs des choix
- 3. Stack globale — tableau récapitulatif
- 4. Justification détaillée par technologie
- 5. Correspondance architecture MVC → code source
- 6. Déploiement et environnement
- 7. Limites connues et évolutions possibles

1. OBJET DU CAHIER D'IMPLÉMENTATION

Le **cahier d'implémentation** explique **pourquoi** chaque technologie a été retenue pour construire DR-DOCSCOL. Il complète le cahier de conception (qui décrit **comment** le système est structuré) en documentant les **choix concrets du code**, leurs **alternatives** et les **raisons du rejet** de ces alternatives.
Ce document sert de référence pour la **soutenance** : répondre aux questions du type **« Pourquoi Next.js ? »**, **« Pourquoi Supabase et pas une autre solution ? »**.

2. PRINCIPES DIRECTEURS DES CHOIX

Principe	Application dans DR-DOCSCOL
Un seul langage	TypeScript côté front et back (Next.js full-stack)
MVP réalisable	Stack connue, hébergement gratuit possible (Vercel + Supabase)
Sécurité par défaut	Auth externalisée (Supabase), ORM typé (Prisma), validation systématique (Zod)
Maintenabilité	Services métier centralisés dans lib/, séparation MVC
Contexte Cameroun	Responsive mobile, consultation QR, emails SMTP accessibles
Traçabilité académique	Migrations Prisma versionnées, documentation dans docs/

3. STACK GLOBALE — TABLEAU RÉCAPITULATIF

Couche	Technologie retenue	Version	Rôle dans DR-DOCSCOL
Framework full-stack	Next.js (App Router)	15.x	Pages, API, Server Actions, SSR
Interface	React + Tailwind CSS + Radix UI	React 19	Composants UI, responsive
Langage	TypeScript	5.x	Typage statique, sécurité compile-time
Base de données	PostgreSQL (Supabase)	—	Données métier relationnelles
ORM	Prisma	6.x	Modèle, migrations, requêtes typées
Authentification	Supabase Auth + @supabase/ssr	—	Sessions, mots de passe, reset email
Validation	Zod	3.x	Schémas entrées API et formulaires
Formulaires	react-hook-form	7.x	Gestion état formulaires côté client
Emails	Nodemailer (SMTP)	8.x	Notifications transactionnelles
Stockage fichiers	Supabase Storage	—	Pièces justificatives duplicata
QR code	qrcode (npm)	1.5.x	QR consultation sur la landing
Hébergement	Vercel	—	Déploiement serverless, CI/CD Git
Tests	Node test runner + tsx	—	Tests unitaires services (tests/)

4. JUSTIFICATION DÉTAILLÉE PAR TECHNOLOGIE

Format : Pourquoi retenu → Alternative(s) → Pourquoi non retenu(s)

4.1 — Next.js 15 (App Router)

Pourquoi retenu

- Framework **full-stack** : pages UI, Route Handlers (app/api/), Server Actions (actions.ts) dans un seul projet.
- **Server Components** : chargement initial rapide pour /dashboard/documents, /admin (données lues côté serveur).
- **App Router** : routage par dossiers (app/dashboard/, app/admin/), middleware centralisé pour la session.
- Écosystème mature, documentation riche, déploiement natif sur Vercel.
- Adapté au MVP académique avec délai limité.

Alternatives envisagées

Alternative	Pourquoi non retenue
Laravel (PHP)	Stack différente front/back ; moins adaptée à une SPA React moderne ; hébergement PHP séparé du front

	React si SPA.
Django / Flask	Python côté serveur + React séparé = deux projets à maintenir ; courbe d'intégration plus longue.
Create React App (SPA seule)	Pas de SSR natif ; API à développer séparément (Express/Fastify) ; plus de code boilerplate.
Remix	Valide techniquement, mais écosystème Vercel/Next.js plus documenté pour le contexte du projet.

Phrase soutenance : *« Nous avons choisi Next.js car il regroupe front, back et déploiement dans une stack TypeScript unique, ce qui accélère le développement d'un portail multi-rôles. »*

4.2 — React 19 + TypeScript

Pourquoi retenu

- **React** : composants réutilisables (components/dashboard/, components/admin/), écosystème UI (Radix, lucide-react).
- **TypeScript** : détection d'erreurs à la compilation ; types partagés avec Prisma (lib/generated/prisma).

Alternatives envisagées

Alternative	Pourquoi non retenue
JavaScript pur	Moins sûr sur un projet avec 25+ pages et règles métier complexes (routage OBC/DECC).
Vue.js / Angular	Compétences et template de départ orientés React/Next.js ; cohérence avec Supabase SSR docs.
PHP templates (Blade)	Peu adapté à une interface riche (dashboard, modales, formulaires multi-étapes duplicata).

4.3 — PostgreSQL + Prisma

Pourquoi retenu

- **PostgreSQL** : modèle **relationnel** naturel (élève → examens → documents → RDV → paiements) ; contraintes d'intégrité ; JSON si besoin futur.
- **Prisma** : schéma déclaratif (prisma/schema.prisma), **migrations versionnées** (20 migrations), client typé, pas de SQL injecté manuellement.
- Hébergé sur **Supabase** : Postgres managé + Auth + Storage dans la même plateforme.

Alternatives envisagées

Alternative	Pourquoi non retenue
MySQL / MariaDB	PostgreSQL mieux intégré à Supabase ; enums et contraintes plus expressifs pour les statuts métier.
MongoDB	Données fortement relationnelles (multiplicités UC, audit, paiements) ; NoSQL moins adapté aux jointures fréquentes.
SQL brut (pg)	Plus verbeux, pas de typage auto, risque d'erreurs sur 15+ modèles.
Drizzle ORM	Valide ; Prisma choisi pour Studio, migrations intégrées et maturité documentation académique.
SQLite	Insuffisant pour déploiement cloud multi-utilisateurs

	concurrents.
--	--------------

Phrase soutenance : *« Prisma nous garantit que le code TypeScript reflète exactement le schéma base, ce qui sécurise les transitions de statut document et les imports CSV. »*

4.4 — Supabase Auth (+ @supabase/ssr)

Pourquoi retenu

- Gestion **mot de passe hashé**, reset email, sessions JWT — **hors de Prisma** (bonne pratique sécurité).
- @supabase/ssr : cookies HttpOnly, refresh session dans middleware.ts, compatible Server Components.
- Lien métier : User.authUserId en Prisma ; contrôle **matricule** + **rôle** avant ouverture session (barrière métier OBC/DECC).

Alternatives envisagées

Alternative	Pourquoi non retenue
NextAuth.js	Configuration plus lourde pour multi-rôles + matricule custom ; Supabase déjà couplé à la BDD.
Clerk / Auth0	Services tiers payants en production ; dépendance externe supplémentaire.
JWT maison + bcrypt	Réinventer la roue ; risque faille sécurité ; temps de développement.
Firebase Auth	Moins aligné avec Postgres/Prisma ; vendor lock-in Google.

Phrase soutenance : *« Les mots de passe ne sont jamais stockés dans notre base métier : Supabase Auth gère l'authentification, Prisma gère le profil et le périmètre OBC/DECC. »*

4.5 — Server Actions + Route Handlers (API REST)

Pourquoi retenu

- **Server Actions** (app/dashboard/actions.ts, app/admin/actions.ts) : formulaires élève/admin sans API REST intermédiaire ; moins de code ; revalidation cache Next.js.
- **Route Handlers** (app/api/) : endpoints pour consultation publique, slots RDV, confirmation agent, webhook paiement — **contrat HTTP explicite** pour appels externes ou tests curl.

Alternatives envisagées

Alternative	Pourquoi non retenue
API REST uniquement	Plus de boilerplate (fetch client, gestion erreurs) pour chaque formulaire interne.
GraphQL	Surdimensionné pour le périmètre MVP ; courbe d'apprentissage.
tRPC	Bon choix TypeScript, mais Server Actions suffisent pour formulaires server-first Next.js 15.

Règle appliquée : Server Actions pour actions **internes** (dashboard, admin) ; API REST pour **public** (/api/public/consultation) et **intégrations** (paiement, agent mobile futur).

4.6 — Zod + react-hook-form

Pourquoi retenu

- **Zod** : schémas partagés serveur/client (`lib/validations.ts`) ; messages d'erreur cohérents ; validation API (`lib/api-utils.ts`).
- **react-hook-form** : performances formulaires longs (duplicata, inscription) ; intégration `@hookform/resolvers/zod`.

Alternatives envisagées

Alternative	Pourquoi non retenue
Validation manuelle	Répétitive sur 25+ écrans ; erreurs oubliées.
Yup	Zod mieux intégré à l'écosystème TypeScript / inférence de types.
HTML5 required seul	Insuffisant côté serveur ; contournable.

4.7 — Tailwind CSS + Radix UI (shadcn-style)

Pourquoi retenu

- **Tailwind** : utility-first, responsive rapide (`md:`, `lg:`), cohérent avec charte ardoise/ambre.
- **Radix UI** : composants accessibles (Dialog, Select, Popover) sans imposer un design lourd.
- **lucide-react** : iconographie légère et uniforme.

Alternatives envisagées

Alternative	Pourquoi non retenue
Bootstrap	Look générique ; personnalisation charte OBC/DECC plus laborieuse.
Material UI (MUI)	Bundle plus lourd ; style Google moins adapté à l'identité institutionnelle.
CSS modules seul	Plus lent à itérer sur 25 pages responsive.

4.8 — Nodemailer (SMTP)

Pourquoi retenu

- Envoi emails transactionnels (disponibilité, RDV, duplicata) via **SMTP configurable** (Gmail, serveur institutionnel).
- Journalisation dans `mail_logs` (succès/échec) pour audit.
- Templates HTML dans `lib/email-template.ts`.

Alternatives envisagées

Alternative	Pourquoi non retenue
SendGrid / Resend API	Dépendance API payante ; SMTP suffit pour MVP et démo.
Supabase Edge Functions email	Moins flexible pour templates métier détaillés.
Pas d'email (notifications in-app seule)	Non conforme au CDC (F-12, F-13 notifications email).

4.9 — Supabase Storage

Pourquoi retenu

- Stockage **pièces justificatives duplicata** (4 fichiers par dossier) — filesystem Vercel **éphémère** (non persistant).
- Bucket duplicata-documents, service lib/duplicata-storage.ts.
- Même compte Supabase que Auth et Postgres.

Alternatives envisagées

Alternative	Pourquoi non retenue
Fichiers locaux sur serveur	Perdus à chaque redéploiement Vercel.
AWS S3 direct	Configuration IAM plus complexe pour MVP académique.
Base64 en PostgreSQL	Mauvaise pratique (taille BDD, performances).

4.10 — Vercel (hébergement)

Pourquoi retenu

- Déploiement **Git push** → **build automatique** ; preview par branche.
- vercel-build : prisma migrate deploy avant build (migrations en production).
- Compatible Next.js 15 sans configuration serveur manuelle.

Alternatives envisagées

Alternative	Pourquoi non retenue
VPS (OVH, DigitalOcean)	Maintenance OS, nginx, SSL manuels ; hors scope temps projet.
Render / Railway	Valides ; Vercel optimisé pour Next.js, tier gratuit suffisant démo.
Hébergement mutualisé PHP	Incompatible avec Node.js / Next.js.

4.11 — Bibliothèque qrcode (consultation rapide)

Pourquoi retenu

- Génération QR côté serveur ou build pour la landing (components/landing/consultation-access.tsx).
- Encode l'URL /consultation — scan téléphone → page publique.

Alternatives envisagées

Alternative	Pourquoi non retenue
Service QR externe (api.qrserver.com)	Dépendance réseau tierce ; données URL exposées.
QR statique image Photoshop	Non régénérable si URL production change (NEXT_PUBLIC_SITE_URL).

4.12 — Rate limiting (lib/rate-limit.ts)

Pourquoi retenu

- Protection **consultation publique** contre énumération de matricules (F-38, BNF sécurité).
- Store mémoire en dev ; extensible Redis en production.

Alternatives envisagées

Alternative	Pourquoi non retenue
Aucun rate limit	Risque abus sur endpoint public.
Cloudflare WAF seul	Non configuré dans MVP ; solution applicative plus portable.
Redis obligatoire dès MVP	Complexité infra ; store mémoire suffit soutenance/démo.

5. CORRESPONDANCE ARCHITECTURE MVC → CODE SOURCE

Couche MVC	Technologie	Dossiers / fichiers
Vue	React + Tailwind	app/**/page.tsx, components/
Contrôleur	Server Actions + Route Handlers	app/**/actions.ts, app/api/**/route.ts, middleware.ts
Modèle	Prisma + PostgreSQL	prisma/schema.prisma, lib/prisma.ts
Services métier	TypeScript pur	lib/document-routing.ts, lib/appointment-service.ts, lib/duplicata-service.ts, etc.
Services externes	Supabase, Nodemailer	lib/supabase/, lib/mail-service.ts, lib/duplicata-storage.ts

6. DÉPLOIEMENT ET ENVIRONNEMENT

Élément	Choix	Justification
Runtime	Node.js ≥ 20	Requis Next.js 15 et Prisma
BDD production	Supabase Postgres (pooler)	Connexion serverless Vercel via Supavisor
Variables d'env	.env / Vercel dashboard	Secrets Supabase, SMTP, INTERNAL_API_SECRET
Migrations	prisma migrate deploy	Schéma reproductible, historique Git
CI locale	npm run build, npm test	Validation avant push

7. LIMITES CONNUES ET ÉVOLUTIONS POSSIBLES

Sujet	État MVP	Alternative future
Paiement duplicata	Simulé applicativement	Orange Money / MTN API certifiée
Rate limit	Mémoire	Redis / Upstash en production
Emails	SMTP Gmail	SendGrid institutionnel
Tests	Unitaires services + manuel	E2E Playwright (déjà en devDependency)
Téléchargement PDF diplôme	Hors périmètre	Stockage sécurisé + watermark